

OccasionWear — Backend Schema, ER Diagram, Scenarios, and Dispute Flows

This document contains the entity-relationship schema (ERD), the detailed entity list (table definitions), example end-to-end scenarios, and dispute-resolution flows for the OccasionWear marketplace backend. All terms are spelled out; where an acronym appears it is written in full with the acronym in parentheses the first time.

1. Entity-Relationship Diagram (ERD)

Below is the ERD expressed in Mermaid syntax. You can paste this code into a Mermaid renderer or into compatible documentation tools to visualize it.

```
erDiagram
    USERS ||--o{ USER_PHONES : has
    USERS ||--o{ ADDRESSES : has
    USERS ||--o{ SUPPLIERS : owns
    SUPPLIERS ||--o{ SUPPLIER_ADDRESSES : has
    SUPPLIERS ||--o{ ITEMS : has
    ITEMS ||--o{ ITEM_PHOTOS : has
    ITEMS ||--o{ LISTINGS : has
    LISTINGS ||--o{ BOOKINGS : has
    BOOKINGS ||--o{ PAYMENTS : has
    BOOKINGS ||--o{ DISPUTES : has
    USERS ||--o{ CONVERSATIONS : participates
    CONVERSATIONS ||--o{ MESSAGES : has
    ITEMS ||--o{ CONSIGNMENTS : can_be
    CONSIGNMENTS ||--|| USERS : owner
    CONSIGNMENTS ||--|| SUPPLIERS : intake_by
    PAYOUTS ||--|| SUPPLIERS : to
    AUDIT_LOGS }o--|| USERS : recorded_by
    FILES ||--o{ ITEM_PHOTOS : used_in
    FILES ||--o{ CONSIGNMENT_DOCS : used_in
```

2. Entities (Tables) and Fields

The core tables for the Minimum Viable Product (MVP) backend are listed below. For each table we include the main fields, purpose, and important notes.

users

- id: BIGINT AUTO_INCREMENT PRIMARY KEY
- email: VARCHAR(255), unique, not null
- password_hash: VARCHAR(255), nullable (nullable for OAuth-only accounts)
- name: VARCHAR(200)

- role: ENUM('customer','supplier_admin','platform_ops','admin','proxy_agent')
- is_email_verified: boolean
- is_phone_verified: boolean
- oauth_provider: VARCHAR(50) nullable
- avatar_path: VARCHAR(1024) local filesystem path
- created_at, updated_at, deleted_at

Purpose: Stores user accounts (customers, supplier managers, platform operations staff, admins).

user_phones

- id: BIGINT AUTO_INCREMENT
- user_id: FK -> users.id
- phone: VARCHAR(30)
- is_verified: boolean
- verification_sent_at, verified_at
- created_at, updated_at

Purpose: Stores multiple phone numbers per user and verification timestamps.

addresses

- id: BIGINT AUTO_INCREMENT
- owner_type: ENUM('user','supplier')
- owner_id: BIGINT
- label, line1, line2, city, region, postal_code, country
- lat DECIMAL, lng DECIMAL
- created_at, updated_at

Purpose: Polymorphic addresses for users and suppliers; supports multiple physical locations for suppliers.

suppliers

- id: BIGINT AUTO_INCREMENT
- user_id: FK -> users.id (manager/owner of this supplier)

- name, slug, description
- visible_contact: masked contact shown in UI
- real_contact: actual phone/email stored securely
- contact_revealed: boolean (access controlled per booking)
- rating, status (pending/active/suspended)
- created_at, updated_at

Purpose: Supplier profile for shops and tailors. A single user can manage multiple suppliers if needed.

items

- id: BIGINT AUTO_INCREMENT
- supplier_id: FK -> suppliers.id (nullable for direct consignments)
- title, sku, size, brand, color, condition (new/excellent/good/fair/poor)
- description, is_consignment boolean, consignment_id nullable
- created_at, updated_at, deleted_at

Purpose: Represents physical garments. Items may be supplier-owned or consigned from an external owner.

files (generic)

- id: BIGINT AUTO_INCREMENT
- owner_type: ENUM('item','consignment','user','booking','other')
- owner_id: BIGINT
- path: VARCHAR(1024) (local filesystem path)
- mime_type, size_bytes, checksum
- uploaded_by: FK -> users.id
- created_at

Purpose: Central table for media: intake photos, listing photos, signed agreements, ID scans, proof-of-delivery photos.

consignments

- id: BIGINT AUTO_INCREMENT

- owner_user_id: FK -> users.id (the person who owns the item)
- supplier_id: FK -> suppliers.id (intake location)
- status: ENUM(draft,intake_pending,active,returned,sold,rejected)
- signed_agreement_path: path to agreement file (local filesystem)
- intake_at, intake_by, notes, created_at, updated_at

Purpose: Tracks consigned items and the signed consignment agreement.

listings

- id: BIGINT AUTO_INCREMENT
- item_id: FK -> items.id
- type: ENUM('rent','sell')
- price: DECIMAL
- deposit_amount: DECIMAL
- pricing_unit: ENUM('per_day','per_event','flat')
- availability_status, available_from, available_to
- cancellation_policy, created_at, updated_at

Purpose: Offers to rent or sell a specific item.

bookings

- id: BIGINT AUTO_INCREMENT
- listing_id: FK -> listings.id
- renter_id: FK -> users.id
- start_date, end_date
- booking_status:
ENUM(pending,deposit_paid,confirmed,picked_up,returned,completed,cancelled,disputed)
- deposit_amount, deposit_paid boolean
- payment_method: ENUM('COD','manual_transfer','card')
- pickup_location_address_id, dropoff_location_address_id
- created_at, updated_at

Purpose: Reservations for listings. Deposit controls contact reveal behavior.

payments

- id: BIGINT AUTO_INCREMENT
- booking_id: FK -> bookings.id (nullable)
- type: ENUM('deposit','refund','payout','fee','damage_charge','supplier_penalty')
- amount: DECIMAL
- method: ENUM('COD','manual_transfer','card','internal_adjustment')
- status: ENUM(pending,completed,failed)
- processed_by: FK -> users.id (who recorded the payment)
- reference, created_at, completed_at

Purpose: Records financial transactions, including manual cash (Cash On Delivery) deposits recorded by staff.

payouts

- id: BIGINT AUTO_INCREMENT
- supplier_id: FK -> suppliers.id
- amount: DECIMAL
- status: ENUM(scheduled,processing,completed,failed)
- hold_until: DATETIME (e.g., completed_at + 7 days)
- transactions: JSON list of booking ids
- created_at, completed_at

Purpose: Batches sent to suppliers. Holds (time windows) allow time for disputes to be raised.

disputes

- id: BIGINT AUTO_INCREMENT
- booking_id: FK -> bookings.id
- raised_by: FK -> users.id
- status: ENUM(open,investigating,resolved,rejected)
- type: ENUM(damage,not_received,fraud,other,damage_consignment)

- claimed_amount: DECIMAL, resolution text, created_at, updated_at

Purpose: Tracks formal complaints and damage claims.

conversations and messages

- conversations: id, subject, booking_id, created_by, created_at

- conversation_participants: conversation_id, user_id, role, muted

- messages: id, conversation_id, sender_id, body, is_redacted boolean, has_contact_flag boolean, attachments JSON, created_at

Purpose: Lightweight chat between renters, suppliers, and ops. Messages are scanned for contact information and redacted if needed.

audit_logs

- id: BIGINT AUTO_INCREMENT

- actor_id: FK -> users.id (who performed action)

- action: VARCHAR

- target_type: VARCHAR

- target_id: BIGINT

- payload: JSON (metadata)

- created_at

Purpose: Immutable record of important actions (deposit recorded, contact revealed, intake completed).

3. End-to-End Scenario

This scenario shows a full flow: consignor gives item to shop, shop lists the item, renter books and pays deposit by Cash On Delivery (COD), pickup, return, and payout scheduling. Each step lists which tables are read or written and what is recorded.

Actors

Carol — consignor (the person who owns the dress and gives it to a shop to sell or rent on her behalf).

Boutique Bliss — supplier (the physical shop that receives the dress and lists it).

Alice — renter (the customer who rents the dress).

Platform operations staff — personnel who record payments, mediate disputes, and schedule payouts.

Step-by-step flow

- Supplier and account onboarding

A shop manager creates a user account. Tables written: users, suppliers, addresses. The supplier profile stores masked contact information and the real contact is stored but not exposed.

- Consignor creates consignment

Consignor registers and uploads a signed consignment agreement and ID scan. Tables written: users, consignments, files.

- Supplier intake and item creation

Shop staff inspects the dress, takes timestamped intake photos (proof of condition), creates an item record and links it to the consignment. Tables written: items, files, consignments (status update), audit_logs.

- Listing creation

Supplier creates a listing with a rental price and required deposit. Tables written: listings.

- Search and listing view

Renter searches; API reads items, listings, files, and suppliers. UI shows masked contact and deposit amount required.

- Booking creation (pending)

Renter creates booking. Tables written: bookings, audit_logs. Deposit is required; until deposit is recorded, supplier's real contact stays hidden to the renter for this booking.

- Deposit recorded (Cash On Delivery)

Because payment is Cash On Delivery (COD), the renter pays the deposit at shop POS. Platform operations staff record the deposit in payments table with method 'COD' and status 'completed'. Booking updated deposit_paid=true and booking_status='deposit_paid'. The system records an audit log and emits a 'deposit_paid' event to reveal contact for that booking only.

- Chat and paste-redaction

Renter and shop exchange messages. If contact info is pasted, the message is flagged and redacted in the user interface and an audit record is written. Tables written: conversations, messages, audit_logs.

- Pickup and proof-of-delivery

Shop marks booking as 'picked_up' and uploads a pickup proof photo. Tables written: bookings (status update), files, audit_logs.

- Return and inspection

Shop inspects the item on return and uploads return photos. Booking status becomes 'returned' or 'disputed' if damage found. Tables written: files, bookings, disputes (if any), audit_logs.

- Payout scheduling

If no dispute, booking is marked 'completed' and a payout record is scheduled for supplier with a hold of 7 days. Tables written: bookings, payouts, audit_logs. A background job will process payouts when the hold expires.

4. Dispute Cases and Resolution Flows

Two common dispute cases are described below, with practical resolution steps and how the database records are used.

Case A — Item owned by shop and damaged by renter

Summary: The shop owns the item. After the renter returns it, the shop finds damage.

- Collect evidence: intake photos (files), pickup proof (files), return photos (files), booking and payment records (bookings, payments), chat logs (messages), and audit logs (audit_logs).

- Shop raises a dispute: create a disputes row linked to the booking and set booking_status='disputed'.

- Compensation flow: automatically deduct up to the deposit amount recorded for the booking. Record this as a 'damage_charge' payment in the payments table (type='damage_charge').

- If claimed amount exceeds deposit, open a secondary claim for the remaining amount and notify the renter to pay the balance; platform operations staff mediate.

- Payouts: exclude the disputed booking from scheduled payouts until resolution; adjustments are kept in payouts records and payments ledger.

- Resolution: if resolved for shop, mark dispute as 'resolved', keep damage_charge payment; if resolved for renter, refund deposit and mark dispute 'rejected'. Record final actions in audit_logs.

Case B — Item consigned by an owner (consignor); renter damages it; shop missed damage at return; consignor later discovers damage

Summary: The item belongs to an external consignor. The consignor gives it to the shop to list. A renter damages the item and the shop fails to notice. Later, when the consignor retrieves the item, they find the damage.

- Key principle: custody and evidence. The party who had custody when damage occurred (usually the renter) is liable. However, the shop has a duty to inspect at handover and return; failure to inspect may make the shop partially or fully liable.
- Required evidence: consignment signed agreement (files), intake photos (files), booking records (bookings), pickup/return photos (files), messages (messages), and audit logs.
- If intake photos exist and show no damage before rental, and return photos show damage after rental, evidence points to renter. Use deposit first to compensate consignor.
- If intake photos are missing or poor quality, evidence is weak. The platform operations staff may hold the shop responsible for negligence per the consignment agreement.
- Compensation steps: create a disputes row (type='damage_consignment') raised by consignor, freeze payouts related to the booking, and run mediation. If shop is liable, record a 'supplier_penalty' payment and deduct from supplier's future payouts. If renter is liable, charge renter's deposit and request additional payment if needed.
- Contractual protections: require mandatory intake photos and a signed consignment agreement to reduce ambiguity; make deposit size proportional to item value.

5. Practical Recommendations (full labels)

- Mandatory timestamped intake and return photos with staff identifier: require capture time and staff user id for each photo upload.
- Signed consignment agreement (Portable Document Format file path) stored and linked to the consignment record.
- Deposit sizing by item value: set deposit at a level that covers expected repair costs for similar items.
- Audit trail for any contact reveal or payment action: always write to audit_logs when contact is revealed or payments are recorded.
- Automated redaction of pasted contact information in messages: detect phone numbers and mask them in the user interface while storing an encrypted copy for audits if legally acceptable.